

**Project ID:** TBD

**Project Source:**

# Project Title

[AI / Software Engineering] Human–AI Data Annotation, Provenance, and Dataset Evaluation Platform

## Project Description and Scope

### Project Overview

Annotation platforms are usually built around the moment of labelling. Serious dataset production is mostly everything else: where the source came from, which guideline version was in force, who annotated and who reviewed, what the AI suggested and whether the human

took it, how disagreement was settled, and what finally entered a release. When that record is missing, a dataset cannot be defended — only shipped.

Arc Intelligence has an existing annotation platform with the core workflow and interface already in place. This project builds the layer above it: a provenance-aware human–AI data production system in which every accepted label can be traced back through the reviews, suggestions, corrections, and guideline versions that produced it.

The goal is not faster annotation. The goal is that a released dataset can answer, for any item in it, *how did this label get here* — and refuse to release when it cannot.

### Project Goals

The system should carry an organisation through one coherent journey:

- **Govern** — organisations, teams, projects, roles, capability-based permissions, and versioned annotation guidelines.
- **Assist** — AI-generated suggestions delivered through a replaceable, typed model interface, supporting human-only, AI-first, and blind-then-reveal modes.
- **Record** — for every item: source and guideline version in force, annotation attempts, and whether the human accepted, modified, rejected, or independently replaced the AI's suggestion. Provenance is captured as it happens, not reconstructed afterwards.
- **Adjudicate** — configurable review, disagreement, and adjudication workflows, with the distinction between an annotator's label, a reviewer's decision, and an adjudicated canonical judgment preserved rather than collapsed into one "final answer" field.
- **Release** — immutable dataset releases with export manifests, gated by checks that detect unreviewed items, missing provenance, and superseded or incompatible versions *before* the release exists.
- **Evaluate** — human-only and AI-assisted annotation compared under repeatable conditions.

A labelling tool with an audit log bolted on will not satisfy the project. Neither will a system in which the release gate can be satisfied by an operator asserting that review happened.

### Scope of Work

Work will span organisation and permission management; source and guideline versioning; the provenance event model; the three annotation modes; reviewer and adjudicator surfaces;

model adapters producing structured suggestions; release validation and export; quality and operational analytics; the evaluation harness; and migration from the existing platform.

## **Working with the Existing System**

The platform is still moving, and very little is frozen for its own sake; a well-argued replacement of almost any part of it is welcome. Silent replacement is not.

Three surfaces are stable by default: the UI style and design language, authentication, and any feature that already works. Changing one is permitted, but the justification comes first and in writing — what is wrong with what exists, what the replacement buys, what it costs — and it is agreed at a review, not discovered in a diff. Rewriting a working subsystem because regenerating it is faster than reading it is the specific failure this project will not accept.

## **Evaluation Harness and Casebook**

Evaluation is a primary deliverable, not a final-week activity. The team will build a reusable harness and roughly 40–60 substantive end-to-end cases supporting repeatable regression testing, including adversarial ones: confidently wrong AI suggestions; genuinely ambiguous items; annotators who disagree and guidelines that do not settle it; source or guideline versions that change after annotation; broken or absent provenance; unreviewed items reaching a proposed release; releases containing superseded items.

Evaluation should measure three things. **Quality** — annotation accuracy, inter-annotator and reviewer agreement, AI suggestion quality and calibration. **Collaboration** — what AI assistance does to human judgment: acceptance, modification and override rates, time per item, review burden, anchoring and automation bias. **Integrity** — provenance completeness, permission and workflow correctness, and whether a release can be reproduced from its manifest.

Dataset quality must not be reduced to a single unexplained score.

## **Expected Outcomes and Deliverables**

- An integrated annotation and dataset-production platform demonstrating the full journey on realistic data
- End-to-end provenance, versioned sources and guidelines, and immutable releases with export manifests
- AI assistance behind a typed model interface, with mock and live modes interchangeable
- A reusable evaluation harness and benchmark casebook with regression support
- Automated, integration, permission, provenance, and end-to-end tests
- Findings documentation: what was tried, what worked, what failed and why — negative findings carry equal weight
- Per-member decision records: significant design and technical choices, alternatives considered, and the reasons
- A technical report covering architecture, evaluation results, limitations, and failure cases, plus a live or recorded end-to-end demonstration

## **Specific Required Knowledge, Skills, and/or Technology**

Students are expected to have strong software engineering ability and to work

independently on a complex, open-ended AI and data platform. The team should collectively demonstrate:

- Ability to read, debug, refactor, and extend unfamiliar codebases, and to build substantial web applications, APIs, and relational data models in Python, TypeScript, or comparable languages
- Understanding of software architecture, permissions, versioning, migrations, testing, and audit history
- Familiarity with large language models, structured outputs, model APIs, or agent systems
- Ability to reason about human–AI interaction rather than treating model output as ground truth — and to preserve conceptual distinctions in a system rather than simplifying them away
- Ability to design rigorous evaluation cases and construct failure cases — proving a system rather than demonstrating it
- Ability to explain trade-offs, limitations, and unresolved risks clearly

Students should expect to work across frontend and backend engineering, data architecture, AI integration, workflow design, and evaluation.

## **Responsible Use of AI Tools**

Arc permits the use of AI tools — coding agents, translation, and similar — at the student's own risk and under the student's full responsibility. If used, the standard is: tools may increase execution capacity, never replace judgment. Every member remains fully responsible for the correctness, security, architecture, and testing of what they submit, must be able to explain and defend any design without appeal to the tool that produced it, and will be asked to walk through the reasoning behind their recorded decisions during reviews.

This applies with particular force to existing code. Directing an agent to regenerate a working subsystem is not a shortcut through the change discipline above — an agent-authored rewrite needs the same prior justification as a hand-written one, and the member who submitted it is expected to defend every line of it.

## **Fields That This Project May Involve**

Artificial Intelligence; Software Engineering; Data Engineering; Human–Computer Interaction; Information Systems; Web Development; Data Science and Analytics.

## **Dataset and Resources Provided by the Client**

The client will provide access to the existing annotation platform, curated datasets and fixtures, example AI interfaces with a deterministic mock, deliverable templates, evaluation guidance, and technical supervision with staged milestone reviews.

Earlier implementation choices may be retained, refactored, or replaced following architectural review.

## **Explicit Non-Goals**

The required scope does not include rebuilding the entire annotation interface, training foundation models, inventing a universal annotation ontology, building a generic workflow engine, supporting every annotation modality, or claiming formal privacy or legal compliance through software features alone.